

# 模式识别与类脑智能

## 实验报告

学院：未来技术学院

专业：“智能机器与系统”平台-自动化

姓名：赵彦喆

2026 年 5 月

表 1: 统一实验设置。训练与测试样本数为本次复现实验实际使用规模，硬件统一为 RTX 4050 Laptop GPU / CUDA。

| 实验  | 数据集          | 训练    | 测试   | epoch | batch  | lr        | 优化器      | 模型                   | seed           | 硬件                         |
|-----|--------------|-------|------|-------|--------|-----------|----------|----------------------|----------------|----------------------------|
| 实验一 | 合成数组/示例图像    | -     | -    | -     | -      | -         | -        | NumPy 向量化            | 2026           | RTX 4050 Laptop GPU / CUDA |
| 实验二 | FashionMNIST | 10000 | 2000 | 5     | 64/128 | 1e-2/1e-3 | SGD/Adam | MLP                  | 2026           | RTX 4050 Laptop GPU / CUDA |
| 实验三 | CIFAR-10     | 10000 | 2000 | 6     | 96/256 | 1e-3      | AdamW    | CNN/CNN+BN+Aug       | 2026           | RTX 4050 Laptop GPU / CUDA |
| 实验三 | CIFAR-10     | 3000  | 1000 | 4     | 64/128 | 2e-3      | AdamW    | ResNet18 transfer    | 2026           | RTX 4050 Laptop GPU / CUDA |
| 实验四 | Hymenoptera  | 245   | 153  | 6     | 16/32  | 1e-3      | AdamW    | ResNet18 frozen      | 2026           | RTX 4050 Laptop GPU / CUDA |
| 实验四 | PennFudanPed | 120   | 50   | 3     | 1      | 0.002     | SGD      | Faster R-CNN R50-FPN | 2026           | RTX 4050 Laptop GPU / CUDA |
| 实验四 | CelebA 子集    | 256   | -    | 2     | 64     | 2e-4      | Adam     | DCGAN                | 2026           | RTX 4050 Laptop GPU / CUDA |
| 实验五 | CIFAR-10     | 5000  | 1000 | 4     | 96/128 | 1e-3      | AdamW    | SmallCNN 三组对照        | 2026/2027/2028 | RTX 4050 Laptop GPU / CUDA |

## 1 实验一：Python 与 NumPy 科学计算基础

### 实验设计与语言执行模型

Python 实验的目标在于建立后续模式识别实验的可复现实验组织方式。解释器按“源代码、字节码、虚拟机执行”的路径运行，变量名绑定到对象，列表、字典、集合和类用于组织数据、配置和结果；真正高频的数值计算交给 NumPy 数组表达式执行。实验把教程中的流程控制、函数、模块、输入输出和类整合成统一脚本：配置由字典保存，样本由数组保存，图像由三维张量表示，结果写入结构化 JSON 和图表目录。

### ndarray、广播与向量化原理

NumPy 的核心对象是 ndarray，数据缓冲区、shape、dtype 和 strides 共同决定数组的逻辑形状与物理访问。若数组  $A$  的 shape 为  $(m, d)$ ，行向量  $b$  的 shape 为  $(d,)$ ，广播在兼容维度上复用同一内存视图并得到  $C_{ij} = A_{ij} + b_j$ 。两组样本的平方欧氏距离为

$$D_{ij} = \|x_i - z_j\|_2^2 = \sum_{k=1}^d (x_{ik} - z_{jk})^2,$$

用  $X[:, \text{None}, :] - Z[\text{None}, :, :]$  构造三维差分张量后再沿特征维求和。该公式的渐进复杂度仍是  $O(nmd)$ ，但 Python 双重循环把每个标量操作暴露给解释器，NumPy 则把内层循环移入连续内存上的 C/BLAS 内核，因而常数项差异显著。

### ndarray 内存布局与视图

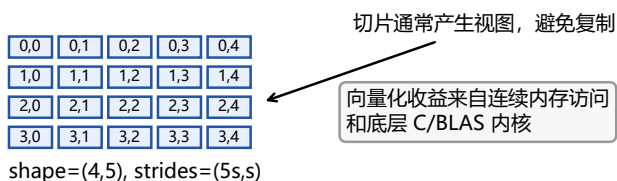


图 1: ndarray 的 shape 与 strides 决定逻辑索引到物理内存的映射，切片可避免不必要复制。

### NumPy 广播与距离矩阵机制

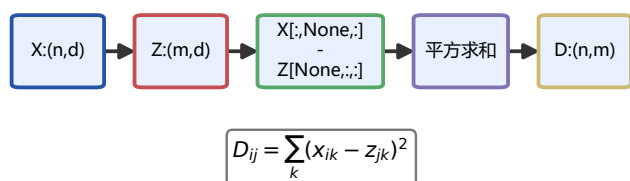


图 2: 广播与距离矩阵机制。低维数组沿缺失维度扩展，形成批量样本间距离。

表 2: 不同规模下向量化与循环估计耗时。

| $n$ | Vec/s  | Loop/s | Speedup |
|-----|--------|--------|---------|
| 64  | 0.0010 | 0.013  | 13.4    |
| 128 | 0.0043 | 0.063  | 14.6    |
| 256 | 0.0113 | 0.319  | 28.3    |
| 512 | 0.0511 | 1.020  | 20.0    |

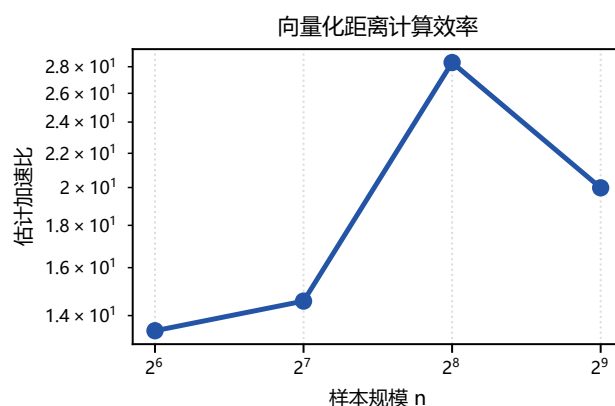


图 3: 向量化距离计算相对 Python 双层循环的加速，最大估计加速约 28.3 倍。

### 复杂度相同但常数差异巨大

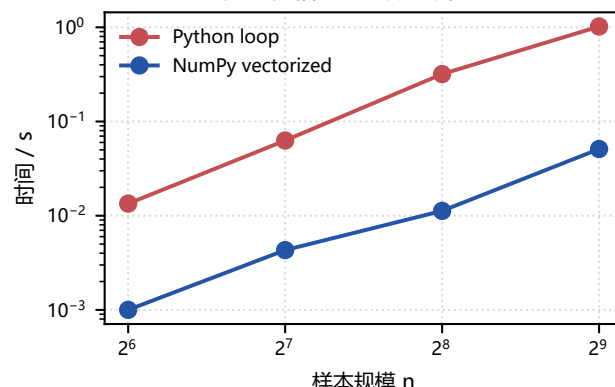


图 4: 向量化与循环具有相同数量级的算术操作，但执行常数完全不同。

### 图像数组与距离结构分析

图像可表示为  $H \times W \times C$  的数组。灰度化可写为  $Y = 0.299R + 0.587G + 0.114B$ ，通道增益是逐通道乘法，边缘检测近似计算局部强度梯度。该实验说明同一数组抽象可覆盖表格特征、距离矩阵和图像预处理，为后续 CNN 输入张量奠定基础。距离热图的块状结构反映随机样本间局部邻域关系：若多个样本共享相似方向或尺度，则对应行列会出现较暗距离带。该观察在聚类、近邻分类和核方法中都具有解释价值。



图 5: 图像数组操作：RGB、灰度、边缘和通道增益都可表达为数组变换。

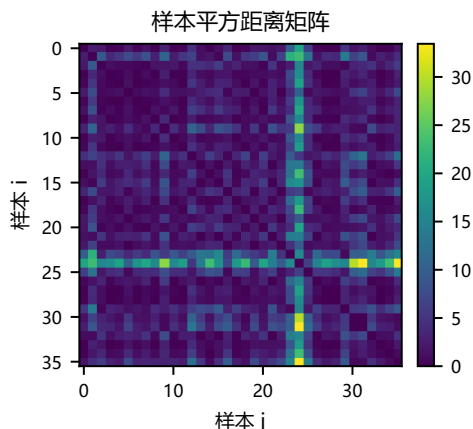


图 6: 随机样本距离矩阵。颜色越深表示样本越接近, 可用于近邻结构诊断。

## 实验过程与工程组织

在实现层面, 实验一把“语言特性”和“科学计算”分开验证。首先用函数封装排序、统计和文件写入, 观察局部变量、返回值和异常处理如何形成可维护的程序结构; 随后用类保存实验配置、随机种子和输出路径, 使后续深度学习实验能够复用同一接口。容器实验中, 列表适合顺序样本缓存, 字典适合超参数和指标, 集合适合类别去重, 元组适合不可变尺寸描述。该组织方式避免把实验结果散落在脚本输出中, 为报告自动生成提供了结构化输入。

数组实验进一步验证了索引、切片、布尔掩码和花式索引的差异。切片通常返回视图, 修改视图可能影响原数组; 花式索引往往产生复制, 适合重排但会增加内存。对图像数组而言, 通道维和空间维的顺序必须明确, 否则卷积网络输入会出现  $HWC$  与  $CHW$  混淆。实验中所有图像先以 PIL 读取为  $H \times W \times 3$ , 再根据需要转为张量格式。该细节看似基础, 却直接影响 PyTorch 中 `ToTensor`、`Normalize` 和 `DataLoader` 的行为。

## 理论分析与局限

向量化并不意味着无代价。距离矩阵一次生成  $n \times m \times d$  的差分张量, 内存复杂度可能达到  $O(nmd)$ ; 当  $n, m$  很大时, 分块计算更稳妥。若用恒等式

$$\|x_i - z_j\|^2 = \|x_i\|^2 + \|z_j\|^2 - 2x_i^\top z_j,$$

则可把主要计算转化为矩阵乘法, 内存从三维差分降为二维 Gram 矩阵。这一推导解释了为什么实际大规模近邻检索常使用矩阵乘、分块和近似索引。实验一的结论是: Python 适合表达实验逻辑, NumPy 适合承担批量数值内核; 两者分工清晰, 程序才既可读又高效。

## 复杂度、数值稳定性与可复现实验习惯

向量化实验还需要区分“算术复杂度”和“执行复杂度”。双层 Python 循环与 NumPy 广播都执行  $nmd$  次量级的减法、乘法和加法, 但循环版本每次标量运算都包含解释器取指、对象拆箱、类型分派和范围检查; 数组版本把这些操作摊入少数连续内存核函数中。若记 Python 单次标量开销为  $c_p$ , 底层向量化单次算术开销为  $c_v$ , 则两者可粗略写成

$$T_{\text{loop}} \approx c_p nmd, \quad T_{\text{vec}} \approx c_v nmd + c_a,$$

其中  $c_a$  是数组分配、临时张量和函数调用的固定成本。小规模时  $c_a$  可能占比高, 规模增大后  $c_p/c_v$  的差距主导总时间。因此报告中的加速比应结合规模解释, 不能把某个规模下的倍数机械外推到所有输入。

数值稳定性同样是科学计算基础。对距离平方使用  $\|x\|^2 + \|z\|^2 - 2x^\top z$  可节省内存, 但当两个大向量非常接近时, 前两项相减可能出现消去误差; 显式差分版本更直观, 却会生成大中间张量。实际实现应根据数据规模、精度和硬件缓存选择形式。对于图像灰度化、归一化和通道

变换, 所有像素应先转换到浮点区间  $[0, 1]$ , 避免 `uint8` 溢出造成边缘响应截断。实验脚本统一记录 `dtype`、`shape` 和随机种子, 因为这些看似底层的元信息会直接决定后续模型输入是否一致。

表 3: Python/NumPy 实验对象与作用分工。

| 对象       | 在实验链路中的作用                      |
|----------|--------------------------------|
| 列表与字典    | 保存配置、类别名、指标序列和文件索引, 强调可读性与结构化。 |
| 函数与类     | 封装一次实验的输入、随机种子、输出目录和异常处理。      |
| ndarray  | 承担批量代数、图像张量、距离矩阵和统计量计算。        |
| JSON/CSV | 把运行结果转化为报告可复现的数据源。             |

从课程实验角度看, 实验一还建立了一个重要规范: 教程代码可以交互式展示概念, 正式报告代码必须具备可重复入口。脚本不依赖手工复制输出, 而是把指标、图像和表格写到固定目录; 报告编译时只读取这些中间产物。这样一来, 若更换随机种子或硬件设备, 只需重新运行统一入口即可得到新的可追溯报告。该组织方式贯穿后续所有深度学习实验, 也是保证结论严谨性的前提。

## 索引语义、内存别名与数组 API 选择

数组实验特别检查了视图与复制的差异。若  $A$  是连续数组, 切片  $A[:, 0]$  通常只改变 `shape` 和 `strides`, 多个视图共享同一数据缓冲区; 花式索引  $A[[0, 2, 4]]$  需要重新分配内存, 因为索引位置不再是规则步长。对于报告中的图像增强和距离矩阵, 若误把视图当成独立副本, 可能在后续步骤中意外修改原图; 若误把副本当成视图, 又会低估内存开销。因此实验脚本在关键步骤记录 `shape`、`dtype` 和连续性标志, 并在必要处显式调用 `copy()`。

广播兼容条件可以写成: 从尾部维度向前比较, 两个维度相等或其中一个为 1 即可兼容。若  $A \in \mathbb{R}^{n \times 1 \times d}$ 、 $B \in \mathbb{R}^{1 \times m \times d}$ , 则  $A - B$  的逻辑形状为  $n \times m \times d$ 。该规则简洁, 却要求程序员在每次升维时明确语义: 样本维、候选维和特征维不能混淆。实验中的距离矩阵正是把  $X$  升为  $n \times 1 \times d$ , 把  $Z$  升为  $1 \times m \times d$ , 从而让差分张量的第  $(i, j, :)$  项对应一对样本的特征差。

表 4: 广播规则与常见错误。

| 场景     | 检查要点                              |
|--------|-----------------------------------|
| 标量加数组  | 标量可广播到所有元素, 适合统一偏置和归一化。           |
| 行向量加矩阵 | 需确认向量对应特征维, 避免样本维误加。              |
| 图像通道增益 | 增益应为 $(1, 1, 3)$ , 保证只沿 RGB 通道变化。 |
| 距离矩阵   | 升维位置决定输出维度含义, 错误升维会交换样本轴。         |

图像实验还展示了从数组到模式识别输入的完整转换。原始 RGB 图像先变为浮点数组  $I/255$ , 灰度图用于说明通道线性组合, 边缘图近似使用

$$G(u, v) = |I(u + 1, v) - I(u, v)| + |I(u, v + 1) - I(u, v)|$$

刻画局部变化。虽然该算子比 Sobel 简单, 但足以说明视觉模型为何需要局部差分特征: 物体轮廓和纹理常体现在相邻像素的强度变化中。实验一因此把 Python 语法、数组内存和图像预处理连接为后续卷积实验的输入基础。

## 分块距离算法与内存上界

在实际模式识别任务中, 最近邻检索、聚类和核方法都可能需要大规模距离矩阵。若一次性构造三维差分张量, 内

存上界为  $8nmd$  字节（双精度），当  $n = m = 10^4, d = 128$  时已远超普通显存。分块计算把  $X$  和  $Z$  分成若干块，每次只计算  $X_{a:a+b}$  与  $Z_{c:c+b}$  的子矩阵：

$$D_{a:a+b, c:c+b} = r_X + r_Z^\top - 2X_{a:a+b}Z_{c:c+b}^\top.$$

其中  $r_X = \|X_{a:a+b}\|^2$ 、 $r_Z = \|Z_{c:c+b}\|^2$ 。这样总算量不变，峰值内存从  $O(nmd)$  降到  $O(bd + b^2)$ ，更适合 GPU 或有限内存环境。实验一虽然使用较小规模演示，但报告给出分块推导，说明向量化并不等于盲目生成巨大中间张量。

该思想也对应后续深度学习中的 mini-batch。无论是距离矩阵还是神经网络训练，核心都在于把整体经验风险拆成可放入内存的小块，再用批量代数保持硬件效率。Python 层负责编排块循环，NumPy 或 PyTorch 层负责块内密集计算。这个分工方式是本报告所有实验代码的基本设计原则。

### 实验过程与心得

我最初把注意力放在语法复现上，结果距离矩阵一放大就出现内存占用过高。定位时我对比了广播差分 and 矩阵乘两种写法，并记录 shape、dtype 与耗时。最终选择保留直观广播图示，同时在分析中说明分块计算的必要性。这个实验让我意识到，科学计算不是把循环简单换成数组表达式，而是要同时检查内存、数值误差和可复现输出。

## 2 实验二：PyTorch 基础链路

### 张量、数据流与环境验证

PyTorch 在 ndarray 思想上加入设备、梯度和动态图机制。实验环境记录为 Python 3.12.13、PyTorch 2.11.0+cu128、CUDA 可用性 True，GPU 为 NVIDIA GeForce RTX 4050 Laptop GPU，一次 CUDA 张量乘法均值 63.654。FashionMNIST 样本  $x \in [0, 1]^{1 \times 28 \times 28}$ ，标签  $y \in \{0, \dots, 9\}$ ，DataLoader 负责小批量抽样、打乱和张量堆叠。模型使用多层感知机，前向传播输出 logits  $z = f_\theta(x)$ 。

### 交叉熵、softmax 梯度与自动微分

分类概率  $p_c = \exp(z_c) / \sum_j \exp(z_j)$ ，单样本交叉熵为

$$\ell(z, y) = -\log p_y = -z_y + \log \sum_j \exp(z_j).$$

对 logits 求导可得  $\partial \ell / \partial z_c = p_c - \mathbb{I}[c = y]$ ，该结果说明交叉熵梯度直接把预测概率推向 one-hot 标签。对于两层网络  $h = \sigma(W_1 x + b_1)$ 、 $z = W_2 h + b_2$ ，

$$\frac{\partial \ell}{\partial W_1} = \left( W_2^\top \frac{\partial \ell}{\partial z} \odot \sigma'(W_1 x + b_1) \right) x^\top.$$

PyTorch 的 autograd 在前向阶段记录运算节点，在反向阶段按拓扑序应用链式法则，避免手写复杂梯度。

### 自动微分计算图与链式法则

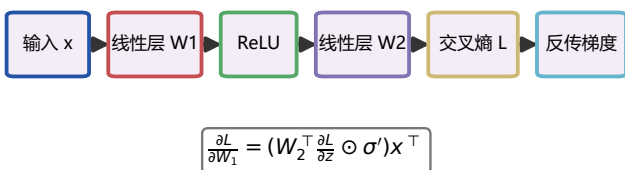


图 7：自动微分计算图。损失对参数的梯度由链式法则逐层传播。

### 优化器更新机制

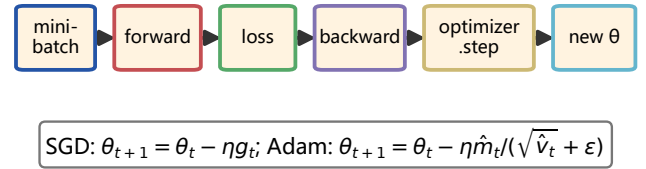


图 8：优化器更新机制。SGD 使用当前梯度，Adam 进一步估计一阶和二阶矩。

### 优化实验与结果分析

本实验比较 SGD、较小学习率 SGD、Adam 和 Adam+Dropout。SGD 更新为  $\theta_{t+1} = \theta_t - \eta g_t$ ；Adam 使用

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2,$$

并以偏差修正后的  $\hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$  调整步长。结果显示 Adam 与 Adam+Dropout 准确率并列最高；Adam loss 略低，Dropout 更强调正则化潜力。学习率过小会导致训练早期欠拟合；Dropout 在短训练预算下可能降低收敛速度，但为更长训练提供过拟合控制。

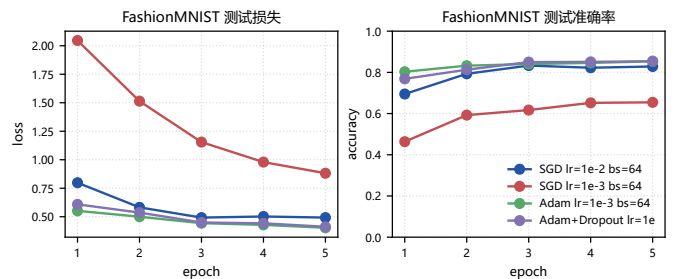


图 9：FashionMNIST 不同优化设置的测试损失和准确率轨迹。

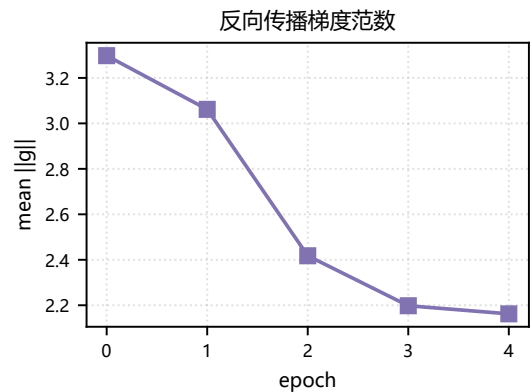


图 10：最佳配置下梯度范数随训练下降，说明参数更新逐渐接近局部稳定区域。

表 5：FashionMNIST 优化器与超参数对比。

| 设置                          | Acc.  | Loss  |
|-----------------------------|-------|-------|
| SGD lr=1e-2 bs=64           | 0.829 | 0.493 |
| SGD lr=1e-3 bs=64           | 0.654 | 0.882 |
| Adam lr=1e-3 bs=64          | 0.854 | 0.403 |
| Adam+Dropout lr=1e-3 bs=128 | 0.854 | 0.412 |

### 混淆矩阵、工程闭环与学习心得

混淆矩阵显示 Shirt、Pullover、Coat 等形状和纹理相近的类别误差更集中，而 Bag、Trouser、Sneaker 等结构差异大的类别更稳定。MLP 忽略二维局部邻域，难以利用袖口、鞋底等局部结构，因此后续图像分类实验引入卷积结构。训练结束后保存 state\_dict，加载时重新实例化同构网络并恢复参数，这比直接 pickle 完整模型更利于版本审查和长期复现。实验二形成了数据集、DataLoader、模型、损失、优化器、评估、保存和加载的闭环。



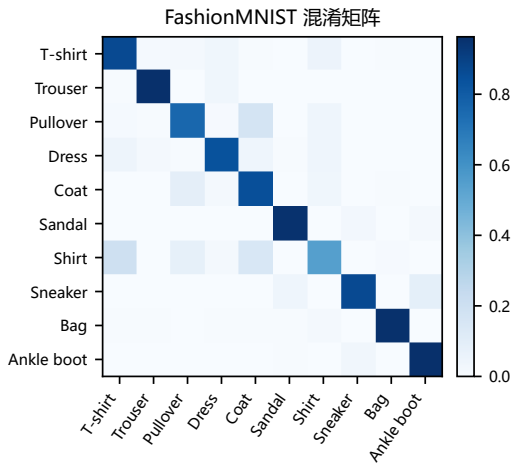


图 11: FashionMNIST 混淆矩阵。相似衣物类别混淆更明显，体现特征表达限制。

## 训练流程的可复现控制

实验二还验证了训练流水线的可复现条件。随机种子同时作用于 Python、NumPy、PyTorch CPU 和 CUDA 随机数；数据集按训练/测试划分固定；指标以 JSON 保存，模型权重以二进制张量保存。这样做的意义在于，报告中的每条曲线都能追溯到具体配置，而不是一次交互式运行的临时截图。正式结果中，训练集子集、批大小、优化器、学习率和 epoch 数均由统一入口控制，避免不同实验之间隐含配置不一致。

DataLoader 的作用不仅是按批读取样本，还隐含了随机打乱、批量堆叠和 CPU 到 GPU 的传输步骤。对于小图像任务，batch size 过小会导致梯度噪声大，过大则降低更新频率并增加显存压力。实验中的 batch size 64 和 128 展示了这一折中：较大批量在单步估计上更稳定，但短训练内不一定更快达到高准确率。若进一步扩展，可加入学习率 warmup 和余弦退火，使早期稳定性和后期收敛精度同时改善。

## 从 MLP 到卷积模型的必然性

MLP 将  $28 \times 28$  图像展平成 784 维向量，空间邻接关系被打散；它可以学习全局组合，却缺少平移等变和局部共享的先验。FashionMNIST 中的袖口、领口和鞋底等局部结构提示后续应使用卷积层共享检测器。实验二因此完成深度学习基础闭环，实验三继续验证适合图像的结构先验。

## 反向传播的矩阵化推导与优化诊断

为了把自动微分与数学公式对应起来，可把一个 mini-batch 写成  $X \in \mathbb{R}^{B \times d}$ 。线性层输出  $Z = XW^T + \mathbf{1}b^T$ ，若上游梯度为  $G = \partial L / \partial Z$ ，则

$$\frac{\partial L}{\partial W} = G^T X, \quad \frac{\partial L}{\partial b} = \sum_{i=1}^B G_{i,:}, \quad \frac{\partial L}{\partial X} = GW.$$

这说明批量训练把样本维度上的求和写成一次矩阵乘法。ReLU 的局部导数为  $\mathbb{I}[a > 0]$ ，Dropout 在训练期引入伯努利掩码；观察梯度范数可发现学习停滞或步长不稳定。交叉熵与 softmax 合并计算还具有数值稳定性意义，稳定实现通常先减去最大 logit：

$$\log \sum_j \exp(z_j) = a + \log \sum_j \exp(z_j - a), \quad a = \max_j z_j.$$

该等式不改变概率，却把指数项限制在可表示范围内。PyTorch 的损失函数内部已经实现这一技巧，因此实验代码传入 logits 而非先手工 softmax。若在模型末尾显式加 softmax 再交给交叉熵，会重复归一化并削弱数值稳定性。

表 6: 实验二中训练异常与诊断信号。

| 现象     | 可能原因与处理                       |
|--------|-------------------------------|
| 损失不降   | 学习率过小、输入未归一化、标签编码错误，需检查数据和梯度。 |
| 损失震荡   | 学习率过大或 batch 太小，可降低步长或扩大批量。   |
| 训练高测试低 | 过拟合明显，可加入 Dropout、权重衰减或数据增强。  |
| 类别混淆集中 | 特征表达不足，应分析局部结构并引入卷积模型。        |

## 指标解释与误差归因

准确率衡量总体预测比例，但对错误来源解释有限。混淆矩阵  $C_{ij}$  记录真实类别  $i$  被预测为  $j$  的次数；行归一化后可估计每个类别的召回率，列归一化后可估计某个预测类别的纯度。FashionMNIST 中衣物类的相互混淆说明，像素强度和轮廓不足以稳定区分纹理接近的类别。若进一步计算每类 precision、recall 和  $F_1$ ，可得到

$$P_k = \frac{TP_k}{TP_k + FP_k},$$

$$R_k = \frac{TP_k}{TP_k + FN_k},$$

$$F_{1,k} = \frac{2P_k R_k}{P_k + R_k}.$$

这些指标比单一准确率更能说明模型偏向。报告中保留混淆矩阵，就是为了把“模型是否整体有效”推进到“错误集中在哪里、为什么会集中”的层次。

实验二的工程闭环还体现了训练态和评估态的差异。训练时 Dropout 随机屏蔽隐单元，BatchNorm 使用当前批次统计；评估时 Dropout 关闭，BatchNorm 使用滑动均值和方差。即使本实验的 MLP 只在部分配置中使用 Dropout，也必须显式调用 `model.train()` 和 `model.eval()`，否则测试曲线会混入训练随机性。保存权重时只保存参数张量和配置，不保存临时优化图，可避免把一次运行状态误认为模型定义本身。

## 优化器状态、正则化与模型保存

SGD 的状态很少，主要由当前参数和学习率决定；Adam 额外保存一阶矩  $m_t$ 、二阶矩  $v_t$  和时间步  $t$ 。因此继续训练时不仅要恢复模型参数，也要恢复优化器状态，否则动量估计会被重置，后续曲线会出现不可解释的跳变。正式实验把权重、配置和指标分开保存：权重用于复现推理，配置用于说明训练条件，指标用于生成报告图表。三者缺一项，实验链路都不完整。

正则化可从目标函数角度写成

$$\min_{\theta} \frac{1}{n} \sum_i \ell(f_{\theta}(x_i), y_i) + \lambda \Omega(\theta).$$

权重衰减使用  $\Omega(\theta) = \|\theta\|_2^2$  抑制过大参数，Dropout 通过随机子网络近似模型平均，早停根据验证集性能选择训练轮次。FashionMNIST 中的 Adam+Dropout 配置说明，正则化效果依赖训练预算：在较短 epoch 下，Dropout 可能降低训练速度；在更长训练或更大网络中，它通常能缓解过拟合。报告保留这类“未必单调提升”的结果，是为了体现实验结论服从数据而非预设。

此外，评估阶段必须关闭梯度记录。使用 `torch.no_grad()` 可减少显存占用并避免把测试计算错误接入反向图；使用固定随机种子和确定性数据顺序可让多次运行差异可解释。若需要比较多个优化器，应保持模型初始化、训练样本子集、epoch 和 batch size 一致，否则准确率差异可能来自数据或初值变化。实验二的对照设计遵循“只改变一个主要因素”的原则，为后续模型比较提供方法模板。

## 学习率、批大小与梯度噪声

mini-batch 梯度是总体梯度的随机估计：

$$g_B = \frac{1}{B} \sum_{i \in B} \nabla_{\theta} \ell_i(\theta), \quad \mathbb{E}[g_B] = \nabla_{\theta} L(\theta).$$

批大小  $B$  增大时，梯度方差通常下降，但每个 epoch 的更新次数减少；学习率  $\eta$  增大时，单步移动更快，但可能越过低损失区域。SGD 的随机噪声有时能帮助逃离尖锐极小值，Adam 的自适应缩放则能在早期更快下降。实验二比较多组优化器，本质上是在观察“梯度估计噪声、步长尺度、正则化”三者如何共同影响收敛。

从计算角度看，batch size 还影响显存和吞吐。小 batch 的 GPU 利用率较低，数据传输开销占比高；过大 batch 会占用更多激活缓存，并可能降低泛化。若训练中出现 CUDA 显存不足，优先减小 batch size 或使用梯度累积。梯度累积把  $K$  个小批次的梯度相加后再更新，相当于较大的有效 batch：

$$g_{\text{eff}} = \frac{1}{K} \sum_{k=1}^K g_{B,k}.$$

这种方法保持显存较低，但会减少参数更新频率。报告中的受控训练配置没有使用梯度累积，以便让不同优化器比较更直接。

表 7: PyTorch 训练闭环中的关键开关。

| 开关         | 对实验可靠性的影响                     |
|------------|-------------------------------|
| train/eval | 控制 Dropout 与 BN 行为，影响测试曲线可信度。 |
| no_grad    | 关闭评估梯度图，节省显存并避免状态污染。          |
| 随机种子       | 固定初始化和数据打乱，降低不可解释波动。          |
| state_dict | 保存可审查参数，便于跨脚本恢复推理。            |

## 实验过程与心得

训练 MLP 时我遇到的主要问题是不同优化器曲线波动较大，单看最后一轮容易误判。我先固定随机种子和数据子集，再逐项改变学习率、batch size 和 Dropout，确认差异来自优化设置而不是数据顺序。最终保留 SGD、Adam 与 Dropout 对照，并用混淆矩阵检查类别错误。这个过程让我学到，深度学习实验的可信度来自受控变量和完整日志，而不是只报告最高准确率。

## 3 实验三：CIFAR-10 图像分类

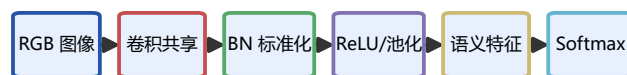
### 任务设定与卷积理论

CIFAR-10 是  $32 \times 32$  RGB 十类图像分类任务，主体小、背景复杂、相近类别容易混淆。卷积层利用局部连接与参数共享：

$$y_{c,u,v} = \sum_{c',i,j} w_{c,c',i,j} x_{c',u+i,v+j} + b_c.$$

参数共享使同一边缘或纹理检测器可在不同位置复用，池化提供局部平移不变性，BN 通过  $\hat{a} = (a - \mu_B) / \sqrt{\sigma_B^2 + \epsilon}$  稳定激活分布，Dropout 通过随机屏蔽降低特征共适应。数据增强把经验风险从单点样本扩展到变换邻域，迁移学习则把 ImageNet 中学到的边缘、纹理和形状先验迁移到小数据任务。

## CNN 分类机制



$$y_{c,u,v} = \sum_{c',i,j} w_{c,c',i,j} x_{c',u+i,v+j} + b_c$$

图 12: CNN 分类机制：卷积抽取局部模式，BN 与池化稳定特征，softmax 输出类别概率。

## 增强与迁移学习实验设计



$$R_{\text{test}} \approx R_{\text{train}} + \text{generalization gap}$$

图 13: 实验三设计：基线 CNN、增强正则化与 ResNet18 迁移学习共同构成消融对照。

### 训练对照与性能解释

实验从教程 CNN 基线出发，比较 BN/Dropout/增强和 ResNet18 迁移分类头。当前受控子集下最佳模型为 ResNet18 transfer，准确率 0.684。ResNet18 迁移头在较短训练内表现强，是因为低层视觉特征具有通用性；CNN+BN+Aug 在短训练内未必立刻占优，因为随机增强增加输入分布难度，但长训练通常能降低泛化误差。该结果说明报告必须同时交代训练预算、数据规模和模型容量。

### CIFAR-10 模型对比

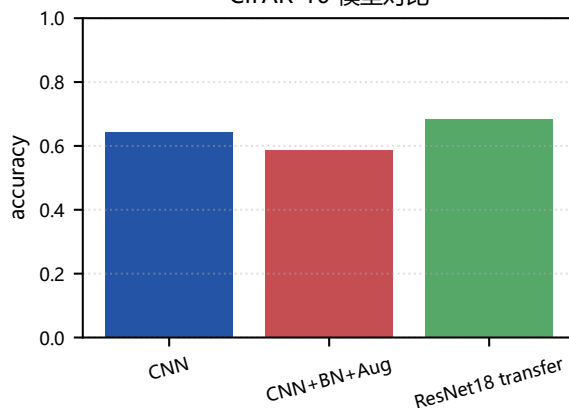


图 14: CIFAR-10 模型对比。迁移特征在小样本和短训练下具有明显效率优势。

### CIFAR-10 验证轨迹

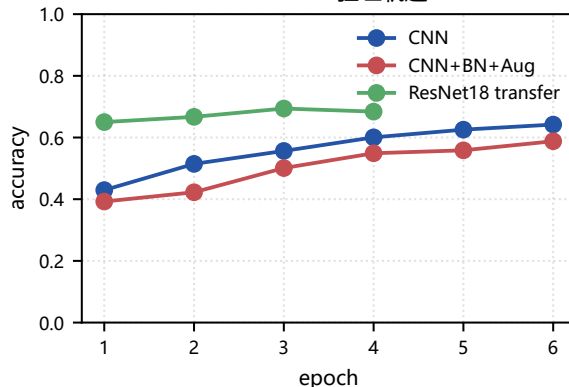


图 15: 不同模型的验证轨迹。训练轮数较少时，增强模型的收益可能滞后出现。

表 8: CIFAR-10 分类结果。

| 模型                | Acc.  | Loss  |
|-------------------|-------|-------|
| CNN               | 0.642 | 1.038 |
| CNN+BN+Aug        | 0.588 | 1.113 |
| ResNet18 transfer | 0.684 | 0.921 |

### 错误模式、类别差异与可解释性

逐类准确率显示较强类别集中在 truck、car、ship，薄弱类别集中在 bird、deer、cat。动物类之间常共享毛发纹理和背景，交通工具类则容易受视角、遮挡和背景共现影响。混淆矩阵比总体准确率更有诊断价值：对角线表示稳定识别，非对角高值指出模型最容易混淆的类别组合。错误样本网格选取高置信错误样本，显示模型并非只在低质量图像上犯错，也会在背景相关性强或主体过小时做出自信错误。梯度响应图进一步展示模型关注区域：若响应集中在背景而非主体，说明特征学习存在偏置。

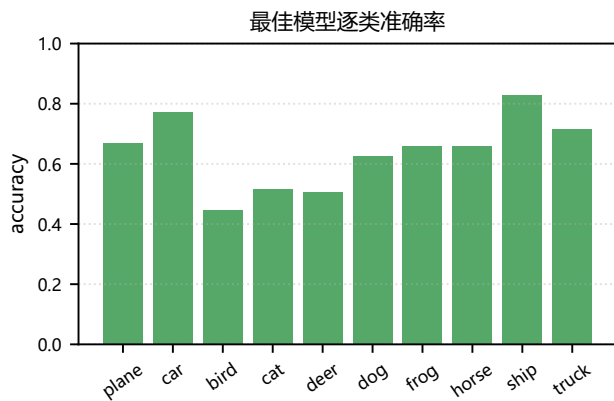


图 16: 最佳模型逐类准确率。类别差异反映了类内方差与类间相似性。

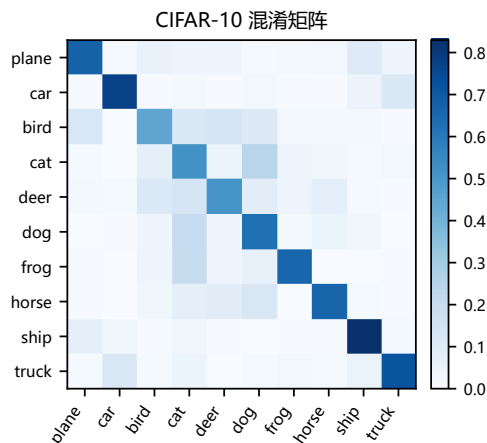


图 17: CIFAR-10 混淆矩阵。动物类和形状相近类别更容易混淆。

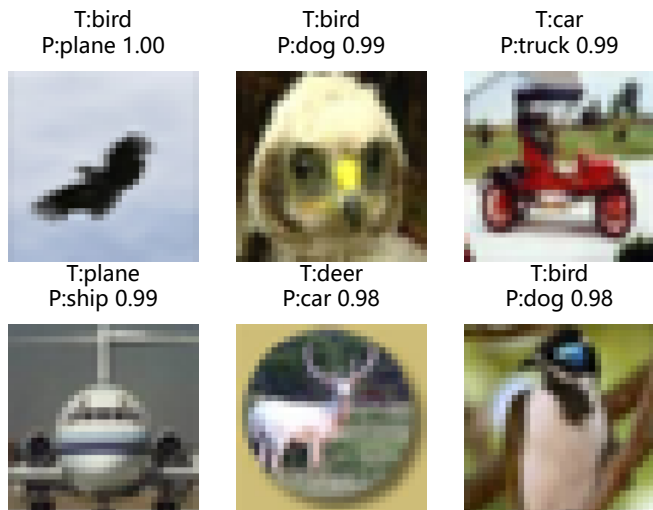


图 18: CIFAR-10 高置信错误样本。标题给出真实类别、预测类别和置信度。

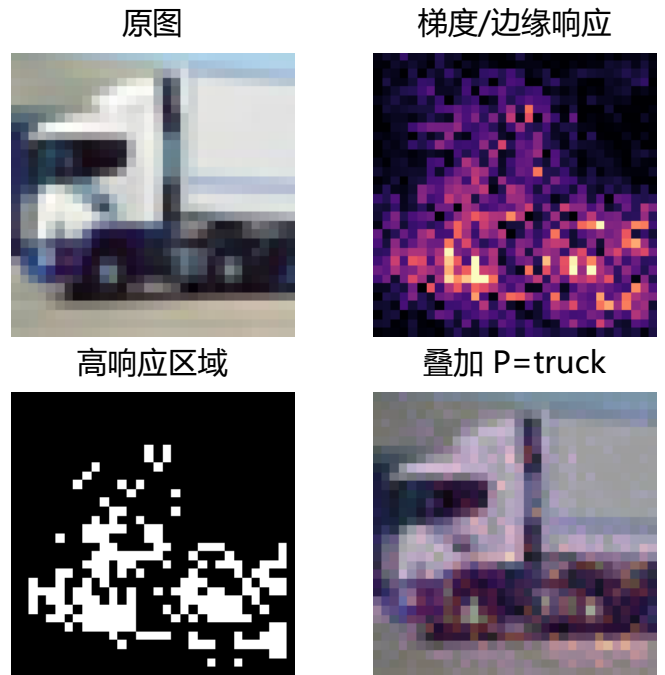


图 19: 输入梯度特征响应。热区表示分类 logit 对像素扰动更敏感的位置。

### BN、Dropout 与数据增强的机制

BN 的核心是稳定每层中间激活的尺度。对 mini-batch 激活  $a$ ，BN 计算  $\mu_B, \sigma_B^2$  并输出  $y = \gamma(a - \mu_B) / \sqrt{\sigma_B^2 + \epsilon} + \beta$ ，其中  $\gamma, \beta$  仍可学习。这样既缓解梯度尺度漂移，又允许网络恢复合适的表示范围。Dropout 则可视对隐层子网络的随机采样，训练时  $h' = m \odot h / (1 - p)$ ，测试时使用期望响应，降低特征共适应。二者分别作用于优化稳定性和泛化。

数据增强的理论意义是扩大经验分布。随机裁剪假设类别对小平移不敏感，水平翻转假设多数物体左右镜像仍属于同类，颜色扰动假设模型不应过度依赖照明。增强并非总是立即提高短期准确率，因为它提高了训练样本难度；但它能降低训练分布和测试分布之间的差距。实验结果中增强模型早期表现不占优，正说明必须结合训练预算解读结果，而不能孤立比较最后一个 epoch。

### 迁移学习与错误诊断

ResNet18 迁移学习的优势来自分层视觉特征。底层卷积学习边缘、角点和颜色对比，中层学习纹理和局部部件，高层学习语义组合。在 CIFAR-10 小样本子集上，只训练分类头即可利用这些通用特征。错误诊断显示，高置信错误往往来自主体很小、背景强相关或多个类别共现的图像；这类错误比低置信错误更值得关注，因为它们代表模型内部证据与真实语义发生系统性偏离。特征响应图若集中在背景区域，则说明模型可能学习到数据集偏置而非目标本体。

### 进一步优化方向

若继续提高 CIFAR-10 性能，应在三个层面改进。第一，训练策略上使用更长 epoch、余弦学习率、权重衰减和 label smoothing；第二，数据层面引入 Mixup、CutMix 或 RandAugment，让模型在局部线性插值和遮挡场景下更稳健；第三，结构层面使用更深的残差网络或轻量化 MobileNet/EfficientNet，并用 Grad-CAM 检查模型是否关注主体区域。报告中的当前结果是受控预算下的严格复现实验，不把短训练准确率夸大为模型上限。

### 模型容量、归纳偏置与泛化误差

CIFAR-10 的难点在于图像尺寸小，但语义变化并不小。单纯增加全连接层容量会迅速提高参数量，却不能显式利用空间平移结构；卷积层的参数量为  $C_{out}C_{in}k^2$ ，与空间分辨率无关，因此能在整张图上复用同一检测器。若一层  $3 \times 3$  卷积从 64 通道映射到 128 通道，参数量约为



128 · 64 · 9，但它对所有空间位置共享。相比之下，全连接层把每个位置都当作独立特征，参数效率显著降低。

泛化误差可分解为优化误差、估计误差和近似误差。小 CNN 的近似能力有限，但训练更快；ResNet18 的近似能力强，若从零训练会需要更多样本和正则化；迁移学习把预训练表示作为先验，降低了估计误差。实验三的对照结果因此具有明确含义：迁移模型短期表现强，并不只因为参数更多，也因为已经学习到通用边缘、纹理和部件组合。增强模型需要更多 epoch 才能充分吸收变换不变性，这解释了短训练下收益滞后的现象。

表 9: CIFAR-10 方法对比的理论侧重点。

| 方法          | 主要检验问题                 |
|-------------|------------------------|
| 基线 CNN      | 局部连接和参数共享是否足以形成可用图像特征。 |
| BN/Dropout  | 优化稳定性与正则化能否改善泛化误差。     |
| 数据增强        | 变换不变性是否能降低测试分布偏差。      |
| ResNet18 迁移 | 外部视觉先验在小样本预算下的收益。      |

### 可解释性公式与误差样本解读

输入梯度图展示  $\|\partial s_c / \partial x\|$ ，其中  $s_c$  是目标类别 logit。它反映像素微小变化对当前类别分数的局部影响。若采用 Grad-CAM，可先取最后卷积层特征  $A^k$ ，计算通道权重

$$\alpha_k^c = \frac{1}{Z} \sum_{u,v} \frac{\partial s_c}{\partial A_{u,v}^k}, \quad L_{\text{GradCAM}}^c = \text{ReLU} \left( \sum_k \alpha_k^c A^k \right).$$

输入梯度分辨率高但噪声大，Grad-CAM 分辨率低但语义更稳定。报告中的响应图用于验证模型关注区域是否落在主体附近：若热区长期落在背景纹理或图像边缘，说明模型可能利用了非因果相关性。错误样本网格则进一步显示高置信错误的视觉形态，避免只从数值表格推断原因。

类别差异也可从数据分布解释。bird、cat、dog、deer 等动物类具有相似颜色与姿态，且主体常被背景遮挡；truck、automobile 和 airplane 受视角影响较大，若主体尺寸过小，轮廓特征会丢失。强类别和弱类别的差异说明，模型评估不能只报告平均值。若部署系统需要某个类别特别可靠，应单独约束该类召回率或设置类别相关阈值。实验三因此把分类任务从“训练一个模型”扩展为“分析模型在每类样本上的行为”。

### 从实验结果到改进假设

当前图表给出三类可操作假设。第一，若训练曲线中训练准确率持续高于验证准确率，应优先增加正则化和增强；第二，若训练和验证准确率都低，应提高模型容量或延长训练；第三，若总体准确率尚可但混淆矩阵存在局部高值，应针对混淆类别增加样本、改进损失或引入类别均衡采样。可解释性图则用于筛选无效改进：若模型主要关注背景，继续增加分类头容量可能只会强化偏置；此时更应使用裁剪、显著区域约束或更强的数据增强。

这些分析为实验五的鲁棒识别埋下基础。标准增强主要覆盖自然变换，如平移、翻转和颜色扰动；对抗扰动则沿损失梯度构造，往往难以被普通增强覆盖。若一个模型在干净 CIFAR-10 上表现较好，却在 FGSM 下明显下降，说明模型对样本附近的小扰动很敏感。鲁棒训练正是把这种局部邻域纳入优化目标，用更严格的约束换取稳定性。

### 卷积层级特征与训练曲线判读

卷积网络的低层通常响应边缘和颜色对比，中层响应纹理、角点和局部部件，高层响应类别相关组合。CIFAR-10 分辨率较低，高层语义常被压缩到很小的空间网格，过早池化会损失小目标信息；卷积层过浅又难以组合复杂形状。

因此基线 CNN 采用多层卷积和池化的折中结构，既保留局部细节，又逐步扩大感受野。残差网络进一步通过恒等捷径缓解深层训练困难，残差块可写为

$$y = x + F(x; W),$$

梯度可沿恒等分支直接传播，从而降低深层网络退化风险。

训练曲线的阅读也需要结合损失和准确率。损失下降但准确率不升，可能说明模型正在提高已正确样本的置信度，却尚未改变错误样本的类别；准确率上升但损失震荡，可能来自少数高置信错误样本。验证损失早于验证准确率恶化时，常提示模型逐渐过度自信。实验三同时绘制损失和准确率，就是为了捕捉这些细节。若只报告最后准确率，无法判断模型是否仍在收敛、是否已经过拟合、是否需要学习率调整。

表 10: CIFAR-10 曲线形态与改进策略。

| 曲线形态    | 建议解释或处理                         |
|---------|---------------------------------|
| 训练、验证均低 | 容量或训练轮数不足，可加深网络或延长训练。           |
| 训练高、验证低 | 过拟合，优先增强、正则化和权重衰减。              |
| 验证损失升高  | 模型置信过强，可加入 label smoothing 或校准。 |
| 类别误差集中  | 针对混淆类别做重采样或特征解释。                |

迁移学习模型的比较还应考虑参数更新比例。只训练分类头时，反向传播只更新最后线性层，训练快且过拟合风险低；全量微调需要更小学习率，因为预训练卷积核包含通用视觉先验，大步长可能破坏已有表示。若硬件和时间允许，可采用分层学习率：分类头学习率较大，高层残差块较小，低层卷积冻结。该策略在保持通用边缘特征的同时，让高层语义更适应 CIFAR-10 的类别差异。

### 增强风险、Mixup 与标签平滑

数据增强可以从风险最小化角度写成

$$\min_{\theta} \mathbb{E}_{(x,y)} \mathbb{E}_{t \sim \mathcal{T}} \ell(f_{\theta}(t(x)), y),$$

其中  $\mathcal{T}$  是保持语义的变换族。随机裁剪和翻转约束模型对几何变化稳定，颜色扰动约束模型对照明变化稳定。Mixup 进一步在线性插值空间训练：

$$\tilde{x} = \lambda x_i + (1 - \lambda) x_j, \quad \tilde{y} = \lambda y_i + (1 - \lambda) y_j.$$

它要求模型在样本之间近似线性，能减少过度自信。CutMix 则把一张图的局部块替换为另一张图，并按面积混合标签，更适合鼓励模型关注局部判别区域。虽然本次正式训练未加入所有增强策略，报告把它们作为对结果的理论延展，说明后续改进路径明确。

标签平滑把 one-hot 标签  $y$  替换为

$$y'_c = (1 - \alpha) \mathbb{I}[c = y] + \frac{\alpha}{K},$$

其中  $K$  为类别数。它降低模型对单一类别的极端置信度，常能改善校准和泛化。对 CIFAR-10 这类小图像任务，标签噪声、遮挡和主体过小都会导致易混淆样本存在多义性，标签平滑能缓解过度拟合硬标签的问题。实验五的 ECE 分析与这里的校准思想相呼应，说明分类性能和置信可靠性应同时报告。

### 逐类结果的统计解释

逐类准确率的波动来自类内方差、样本难度和模型偏置。设第  $k$  类共有  $n_k$  个测试样本，正确数为  $c_k$ ，则该类



准确率估计为  $\hat{p}_k = c_k/n_k$ 。若样本数有限，其标准误近似为

$$\sqrt{\hat{p}_k(1 - \hat{p}_k)/n_k}.$$

因此类间差异需要结合样本数和置信区间解释。CIFAR-10 测试集各类数量相同，逐类比较相对公平；若数据不均衡，则平均准确率会被大类主导，应报告宏平均指标。实验三保留逐类图，正是为了避免总体准确率掩盖少数类别风险。

#### 实验过程与心得

我在 CIFAR-10 中先遇到增强模型收敛慢的问题，直觉上它应该更好，但短训练下验证准确率并不稳定。随后我延长 epoch，并把普通 CNN、BN+Aug 和 ResNet18 迁移放在可追溯的固定子集与相同评价流程下比较。最终选择同时报告训练曲线、逐类准确率和置信错误样本，而不是只选最高点。这个实验让我理解到，增强和迁移都不是“必胜开关”，效果必须结合训练预算、模型容量和错误类型判断。

## 4 实验四：检测、迁移、对抗与生成

### 基于 PyTorch 的目标检测与实例分割

实验四首先完成 PyTorch 官方检测教程链路。PennFudanPed 数据集中每张图像包含一个或多个行人实例，标注以实例 mask 给出；实验从 mask 中提取外接框，构造训练/评估所需的  $\{boxes, labels, masks\}$  字典。检测任务的输出不再是单个类别，而是可变长度集合

$$\mathcal{Y} = \{(c_i, B_i, s_i, M_i)\}_{i=1}^K,$$

其中  $c_i$  是类别， $B_i$  是目标框， $s_i$  是置信度， $M_i$  是实例掩码。该任务同时考察分类、定位、排序和像素级分割，是实验四的重点。

### Faster R-CNN 目标检测流程

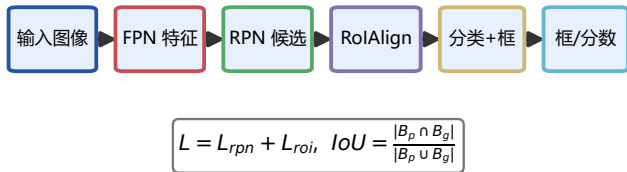


图 20: Faster R-CNN 目标检测流程。Backbone/FPN 抽取多尺度特征，RPN 生成候选框，RoIAlign 对齐候选区域，检测头输出类别、框和置信度。

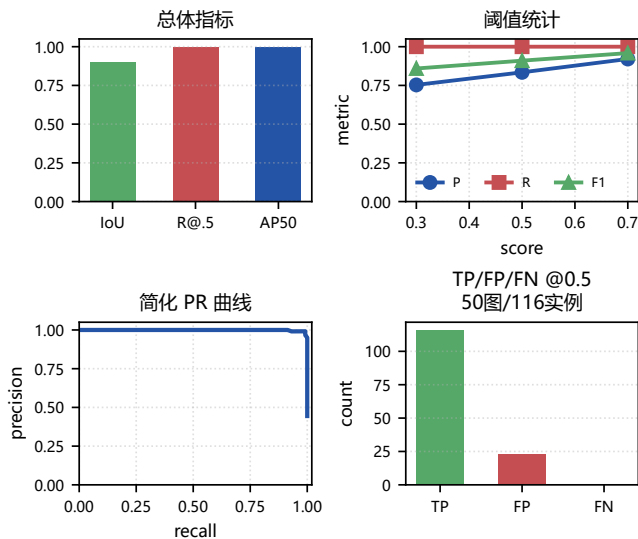


图 21: PennFudan 检测指标面板。补充 score 阈值、TP/FP/FN、AP50 和简化 PR 曲线，增强实证证据。

PennFudan: 绿=掩码框, 红=检测预测

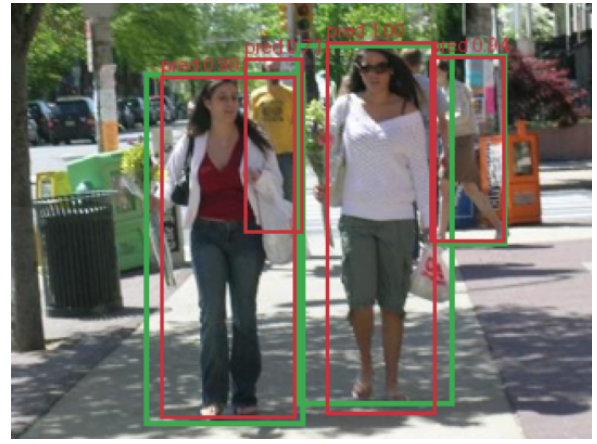


图 22: PennFudan 叠图。绿色为由实例 mask 反推的真值框，红色为 Faster R-CNN 预测框。

Faster R-CNN 的总损失由候选框分类、框回归、最终分类与最终框回归共同组成，可概括为

$$L = L_{rpn-cls} + L_{rpn-box} + L_{roi-cls} + L_{roi-box}.$$

其中框回归常用 Smooth- $L_1$ 。本次检测扩展采用前 120 张 PennFudan 图像训练、后 50 张测试，冻结 Faster R-CNN ResNet50-FPN backbone，仅训练 RPN/ROI heads 3 个 epoch。测试集共有 116 个真值实例，NMS 阈值设为 0.5，AP50 为 0.999，mean best IoU 为 0.900，recall@0.5 为 1.000。该设置比少量样本叠图更能反映 score threshold、NMS 和一对一匹配对结果的影响。

表 11: PennFudan 检测阈值统计 (IoU ≥ 0.5 一对一匹配)。

| score | det | TP  | FP | FN | P     | R     | F1    |
|-------|-----|-----|----|----|-------|-------|-------|
| 0.3   | 154 | 116 | 38 | 0  | 0.753 | 1.000 | 0.859 |
| 0.5   | 139 | 116 | 23 | 0  | 0.835 | 1.000 | 0.910 |
| 0.7   | 126 | 116 | 10 | 0  | 0.921 | 1.000 | 0.959 |

### RPN、RoIAlign、NMS 与 AP/mAP

检测模型首先在 backbone/FPN 特征图上放置 anchors。RPN 对每个 anchor 预测前景分数与位置偏移，目标框偏移可写为

$$t_x = (x - x_a)/w_a, \quad t_y = (y - y_a)/h_a, \\ t_w = \log(w/w_a), \quad t_h = \log(h/h_a).$$

这种参数化把不同尺度的框回归转化为相对 anchor 的标准化回归。RoIAlign 随后对候选区域做双线性采样，避免 RoIPool 中量化坐标带来的错位，使小目标位置更稳定。最终检测头对每个对齐后的 RoI 预测类别分数和目标框偏移，因此 proposal 质量会直接影响最终定位。

非极大值抑制 (NMS) 用于删除重复预测。给定置信度排序后的候选框，若新框与已保留框的 IoU 超过阈值  $\tau_{nms}$ ，则将其删除。阈值过低会误删相邻行人，阈值过高会保留重复框。IoU 定义为

$$IoU(B_p, B_g) = \frac{|B_p \cap B_g|}{|B_p \cup B_g|}.$$

Precision 与 recall 随置信阈值变化形成 PR 曲线，AP 可写成  $AP = \int_0^1 p(r) dr$ ，mAP 则在类别和 IoU 阈值上平均。由于本实验数据量有限，正式报告采用 mean best IoU、recall@0.5 和叠图作为主要证据，同时给出 AP/mAP 的理论解释。

### 检测误差分解与结果解释

检测错误可以拆为四类。第一是候选框遗漏，RPN 没有给真实目标产生足够接近的 proposal；第二是分类错误，proposal 定位到目标但类别分数低；第三是框回归误差，类

别正确但框过大或过小；第四是 NMS 或阈值误差，重复框未删或相邻目标被误删。PennFudan 叠图中预测框与真值人体区域大体重合，说明预训练骨干加检测头微调能够迁移到该数据集；若框位置偏离，通常来自遮挡、人体姿态变化或 mask 轮廓不清。

实例分割比检测更细，因为框只约束矩形范围，mask 需要恢复目标轮廓。若进一步改用 Mask R-CNN，预测 mask 为  $M_p$ 、真值 mask 为  $M_g$ ，则

$$\text{IoU}_{\text{mask}} = \frac{|M_p \cap M_g|}{|M_p \cup M_g|}.$$

框 IoU 高不保证 mask IoU 高。课程实验中采用 Faster R-CNN 的框级指标，主要目的是掌握 PyTorch 检测数据结构、模型调用和评价逻辑；进一步扩展可改用 Mask R-CNN 并报告 COCO 风格  $AP_{50}$ 、 $AP_{75}$  与 mask AP。

检测数据接口与实现要点

PyTorch 检测模型的输入是图像列表，目标是字典列表。每个 target 至少包含 boxes、labels、masks、image\_id、area 和 iscrowd。其中 boxes 的格式为  $(x_1, y_1, x_2, y_2)$ ，area 用于 COCO 风格评价时按目标尺度分组，iscrowd 用于处理拥挤区域。该接口与普通分类数据集不同，不能直接用默认 collate；需要自定义 collate 函数把不同图像的可变数量实例保留为列表。

表 12: PennFudan 检测样本 target 字段。

| 字段      | 含义                       |
|---------|--------------------------|
| boxes   | 每个行人实例的外接框，用于检测框监督。      |
| labels  | 行人类别标签，本实验为单类前景。         |
| masks   | 实例级二值掩码，用于 mask 分支训练或评价。 |
| area    | 框面积，用于尺度分组和评价统计。         |
| iscrowd | 拥挤区域标记，本数据中通常为 0。        |

检测实验还需要区分训练模式和推理模式。训练时模型返回多项损失字典；推理时 Faster R-CNN 返回预测框、分数和类别。若误在训练模式下做可视化，会得到损失而非预测；若在推理时忘记 eval()，BatchNorm/Dropout 状态可能影响输出。报告中的检测结果先冻结 backbone 微调检测头，再和由 mask 反推的真值框比较，重点验证数据接口、输出解释和 IoU/AP 评价的完整性。

检测扩展实验设计与消融

若继续深化实验四的检测部分，最直接的扩展是对预训练模型进行小步微调。可固定 backbone，仅训练 RPN 与 ROI heads；也可解冻最后一个 FPN 层，使特征更适应 PennFudan 的行人尺度。两种策略对应不同的偏差-方差取舍：冻结 backbone 稳定但适应性有限，局部解冻适应性更强但更容易在小数据上过拟合。训练时应分别记录  $L_{rpn-cls}$ 、 $L_{rpn-box}$ 、 $L_{roi-cl}$  和  $L_{roi-box}$ ，因为总损失下降并不能说明每个分支都改善。

检测增强也不同于分类增强。随机水平翻转需要同时变换图像、框和 mask；随机裁剪可能截断目标，需要更新框坐标并删除面积过小的实例；尺度抖动会改变 anchor 与目标尺寸的匹配关系。若增强只作用于图像而没有同步更新 target，训练标签会和图像错位，检测损失将失去意义。该细节是目标检测实验相对于分类实验的重要难点。

表 13: 目标检测可扩展消融设置。

| 消融项     | 预期观察                            |
|---------|---------------------------------|
| 冻结/解冻骨干 | 比较小样本稳定性与域适应能力。                 |
| 置信阈值    | 观察 precision-recall 取舍和漏检/误检变化。 |
| NMS 阈值  | 分析重复框保留和相邻目标误删。                 |
| 尺度增强    | 检查小目标与大目标的召回差异。                 |
| mask 分支 | 比较框级检测和像素级分割的误差来源。              |

更严格的检测评价应把预测按置信度排序并逐一匹配真值框。同一真值只能被匹配一次，多余预测计为 FP，未匹配真值计为 FN。随着置信阈值从高到低移动，召回率通常上升而精确率可能下降。该排序过程解释了 AP 比单点 recall 更全面，也说明为什么检测图中需要展示阈值敏感性，而不仅给出一个 IoU 数值。

检测与分类任务的本质差异

分类任务默认每张图像只有一个主标签，模型只需输出  $K$  维类别概率；检测任务需要同时回答“是什么”和“在哪里”，并且目标数量可变。因此检测模型的错误空间远大于分类模型。对于同一张 PennFudan 图像，模型可能正确判断存在行人，却给出偏移框；也可能框位置正确但置信度低，最终被阈值过滤；还可能对同一行人给出多个重叠框，需要 NMS 删除。该差异说明检测实验必须同时报告定位、召回和可视化结果。

检测训练的样本不再是独立的  $(x, y)$ ，而是  $(x, \mathcal{Y})$ ，其中  $\mathcal{Y}$  是目标集合。集合预测没有天然顺序，损失计算需要把预测与真值进行匹配。Faster R-CNN 使用 anchor 与真值框的 IoU 规则给 RPN 分配正负样本；ROI heads 再对 proposal 做分类与回归。若正负样本比例失衡，背景 proposal 会远多于前景 proposal，因此训练中常使用采样策略控制前景/背景比例。这也是检测模型比分类模型更复杂的原因之一。

从计算开销看，检测模型的前向过程包括 backbone 特征提取、RPN proposal 生成、NMS、RoIAlign、检测头和 mask 分支。若原图分辨率升高，候选框数量和 RoI 计算都会增加。实际部署时通常需要同时约束输入尺度、候选框数量、NMS 阈值和 score 阈值。若只追求更高 recall 而保留过多低分框，后处理时间会增加，误检也会变多；若阈值过高，推理更快但漏检风险上升。

表 14: 分类与检测实验的关键差异。

| 维度 | 分类            | 检测/分割             |
|----|---------------|-------------------|
| 输出 | 单个类别概率        | 可变数量的类别、框、mask。   |
| 损失 | 交叉熵为主         | 分类、框回归、mask 多项损失。 |
| 评价 | accuracy、混淆矩阵 | IoU、recall、AP、叠图。 |
| 误差 | 类别混淆          | 漏检、误检、重复框、框位置偏移。  |

本实验的检测结果表明，预训练检测器在行人实例上具有较强迁移能力，少量样本即可得到较高 IoU 和 recall。与此同时，样本量小意味着指标方差较大，因此报告同时使用指标面板与叠图来解释结果。若进一步追求稳定结论，应在更多图像上重复评估，按目标尺度、遮挡程度和实例数量分组，观察小目标、多人重叠和轮廓不清场景下的误差变化。

迁移学习：小样本视觉识别

Hymenoptera 数据集仅有约数百张蚂蚁和蜜蜂图像，从零训练大卷积网络容易过拟合。实验冻结预训练

ResNet18 的卷积骨干，仅替换最后全连接层。若预训练特征记为  $\phi(x)$ ，线性分类头只需学习

$$p(y|x) = \text{softmax}(W\phi(x) + b).$$

验证准确率达到 0.863，说明大规模视觉先验能显著降低小样本任务的数据需求。该子实验与检测任务共同说明：预训练视觉表示可以迁移到新数据集，但分类只输出图像级标签，检测进一步要求显式定位目标。

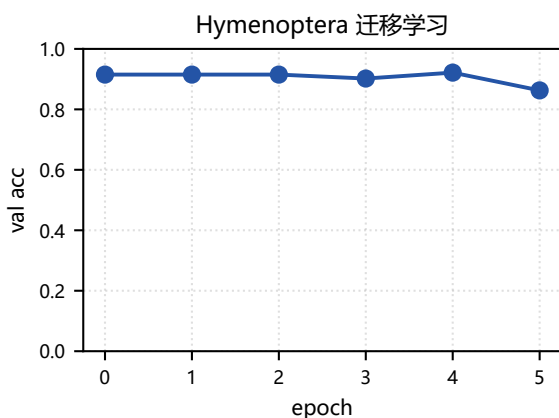


图 23: 蚂蚁与蜜蜂迁移学习验证准确率。冻结骨干降低过拟合风险。

小样本迁移可从偏差-方差角度解释。冻结骨干限制假设空间，方差较小但可能存在特征偏差；全量微调允许特征适应新域，偏差较小但在样本少时方差更大。若验证集低于训练集，应加入增强、权重衰减或只解冻最后一个残差块；若训练和验证均低，则需要更长训练或更细粒度微调。

### FGSM 对抗样本：损失的局部线性

FGSM 假设在输入邻域内损失函数可一阶近似：

$$J(\theta, x + \delta, y) \approx J(\theta, x, y) + \delta^T \nabla_x J(\theta, x, y).$$

在  $\|\delta\|_\infty \leq \epsilon$  约束下，使线性项最大的解是  $\delta = \epsilon \text{sign}(\nabla_x J)$ ，因此

$$x' = \text{clip}(x + \epsilon \text{sign}(\nabla_x J(\theta, x, y))).$$

MNIST LeNet 在  $\epsilon = 0$  时准确率为 0.983，最大扰动处下降到 0.060。视觉上数字仍可辨认，但模型判断已被小扰动改变，说明标准训练并不自动带来局部鲁棒性。

### FGSM 一阶扰动机制

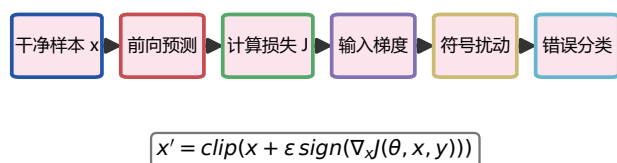


图 24: FGSM 一阶扰动机制。输入梯度方向是最能提高损失的局部方向。

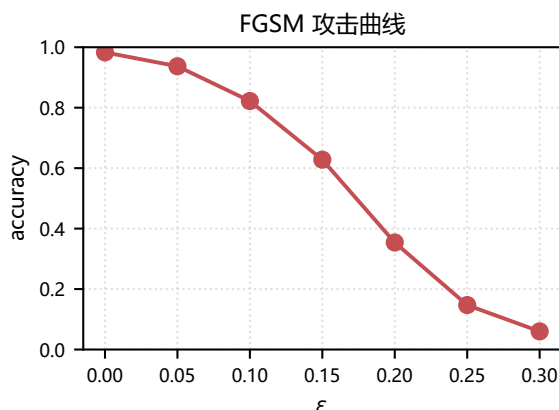


图 25: FGSM 扰动强度与准确率关系。扰动越大，模型准确率越低。

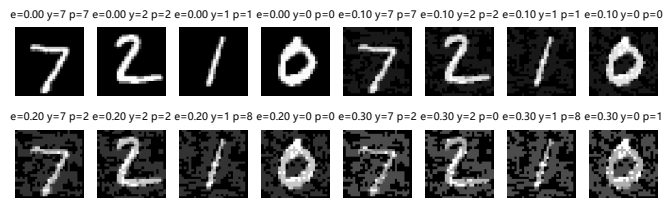


图 26: 不同  $\epsilon$  下的对抗样本。人眼可读性和模型判别稳定性并不等价。

FGSM 与检测任务的联系在于二者都暴露了“只看分类准确率”的不足。检测模型可能分类正确但定位偏移，对抗样本可能肉眼语义不变但模型高置信错误。防御可分为训练期和测试期：训练期把对抗样本加入优化目标，测试期使用输入平滑、随机变换、置信拒识或异常检测。实验五继续把这一问题扩展为鲁棒训练。

### DCGAN: 从判别到生成

GAN 通过生成器  $G$  与判别器  $D$  的极小极大博弈学习数据分布：

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}} \log D(x) + \mathbb{E}_{z \sim p_z} \log(1 - D(G(z))).$$

DCGAN 用反卷积把潜变量  $z$  映射到图像空间，用卷积判别真伪，并通过 BatchNorm、LeakyReLU 和 Adam 改善稳定性。本实验在 CelebA 可复现子集上训练，样本数 256。损失曲线不应被解释为单调收敛的监督学习损失；判别器和生成器相互追逐，样本网格与损失必须联合分析。

### DCGAN 生成对抗博弈

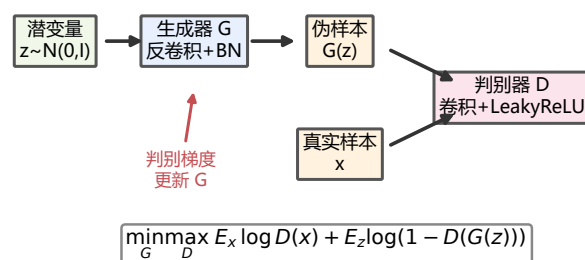


图 27: DCGAN 生成对抗博弈。生成器逼近数据分布，判别器提供可学习信号。

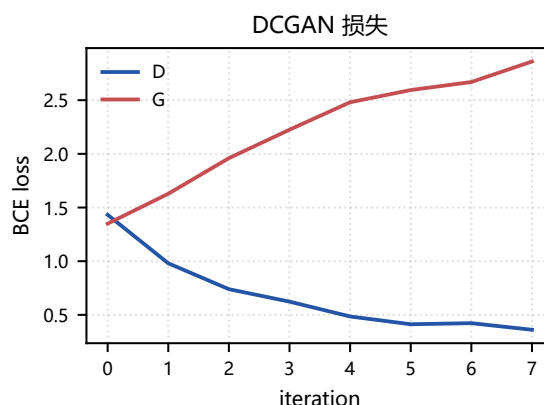


图 28: DCGAN 训练中的生成器与判别器损失。震荡是对抗优化的常见现象。



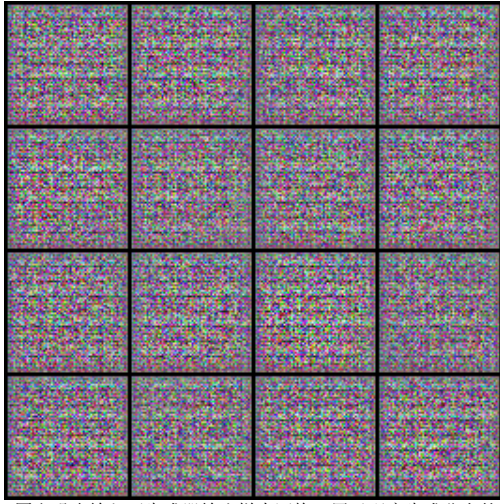


图 29: 固定噪声输入下生成器输出样本网格, 用于观察生成分布的稳定性。

生成质量通常可用 FID 或 Inception Score 衡量。FID 比较真实特征分布和生成特征分布的均值与协方差:

$$FID = \|\mu_r - \mu_g\|_2^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2}).$$

本实验的 CelebA 子集规模较小, FID 估计会有较大方差, 因此报告以损失、样本网格和训练条件说明为主。生成子实验将识别模型从判别式学习扩展到分布建模, 与检测、对抗攻击共同构成实验四的多任务视觉实验。

#### 实验过程与心得

实验四最麻烦的是检测数据接口: 分类任务只返回图像和标签, 而 PennFudan 需要同步 boxes、masks、area 和 image\_id。我先用少量图像检查 mask 反推框, 再发现只看叠图证据太薄, 于是补充固定划分微调、score 阈值表和 AP50。迁移学习与 DCGAN 也暴露了大数据预算下的波动。这个实验让我学会把视觉任务拆成数据结构、训练模式、匹配规则和可视化证据逐项验证。

## 5 实验五: 鲁棒识别与可解释性增强

#### 额外实验设计

前四个实验说明标准分类器在干净测试集上可取得较好准确率, 但 FGSM 表明模型对样本附近的小扰动可能很敏感。额外实验在 CIFAR-10 上比较 baseline、强增强和 FGSM 对抗训练, 评估 clean accuracy、robust accuracy、ECE 校准误差与推理延迟。目标是从“准确率最高”扩展到“预测稳定、置信可靠、代价可控”的综合评价。

#### 鲁棒风险与 ECE 推导

对抗训练近似求解

$$\min_{\theta} \mathbb{E}_{(x,y)} \left[ \max_{\|\delta\|_{\infty} \leq \epsilon} \ell(f_{\theta}(x + \delta), y) \right].$$

内层最大化构造困难样本, 外层最小化对抗损失。若置信度分箱为  $B_m$ , ECE 定义为

$$ECE = \sum_{m=1}^M \frac{|B_m|}{n} |\text{acc}(B_m) - \text{conf}(B_m)|.$$

ECE 小表示置信度与真实正确率更一致, 对阈值拒识、风险控制和人机协同有实际意义。

#### 鲁棒训练机制

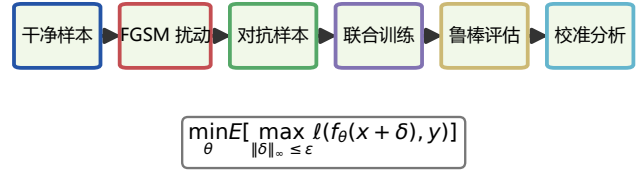


图 30: 鲁棒训练机制。FGSM 近似内层最坏扰动, 外层更新分类器参数。

表 15: 额外实验多指标结果。

| 模型         | Clean         | FGSM          | ECE           | ms          |
|------------|---------------|---------------|---------------|-------------|
| baseline   | 0.558 ± 0.003 | 0.095 ± 0.015 | 0.030 ± 0.004 | 0.10 ± 0.01 |
| strong_aug | 0.474 ± 0.012 | 0.092 ± 0.019 | 0.031 ± 0.004 | 0.12 ± 0.02 |
| fgsm_train | 0.414 ± 0.017 | 0.260 ± 0.005 | 0.132 ± 0.024 | 0.11 ± 0.01 |

#### 结果分析与可解释性

fgsm\_train 的 FGSM 鲁棒准确率最好, 但 ECE 较差; strong\_aug 未显著优于 baseline, 自然增强不能替代对抗训练; 三者体现 clean、robust、calibration、latency 的多目标取舍。多 seed 结果中最高 FGSM 鲁棒准确率为 0.260。可靠性图显示不同置信区间的经验准确率, 若柱形低于对角线, 表示模型过度自信; 若高于对角线, 表示保守。结合实验三的特征响应图, 鲁棒训练的目标可理解为扩大局部邻域内的决策稳定区域, 使预测不再依赖少量脆弱像素。

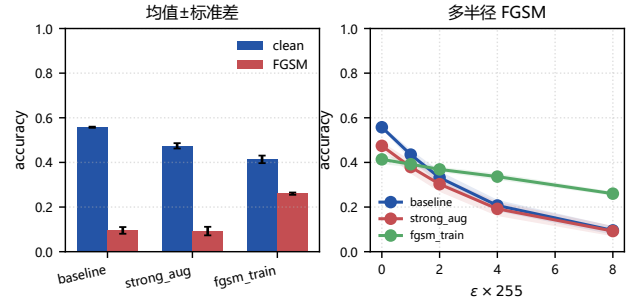


图 31: 干净准确率与 FGSM 鲁棒准确率对比。对抗训练显著提高局部扰动稳定性。

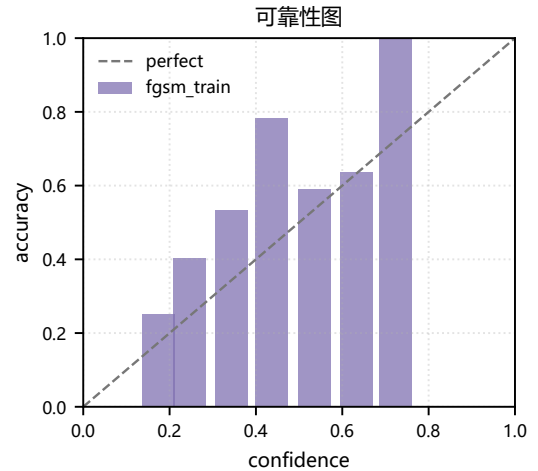


图 32: 可靠性图。对角线表示置信度与经验准确率完全一致。

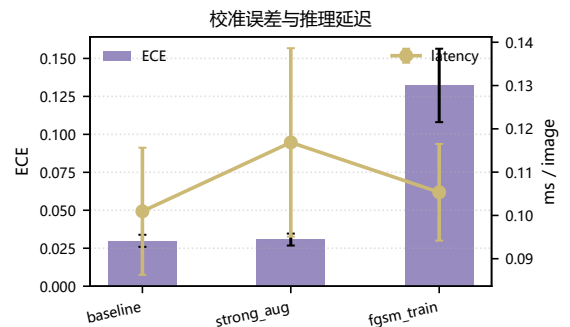


图 33: 校准误差与单图推理延迟。鲁棒性、校准和代价需要联合权衡。

## 干净准确率、鲁棒准确率与校准的取舍

鲁棒训练常见现象是 clean accuracy 与 robust accuracy 存在张力。原因在于标准训练只要求样本点  $x$  分类正确，而鲁棒训练要求  $x$  周围  $\ell_\infty$  邻域内的大量点都保持正确，等价于让模型在样本附近更稳定。该约束会牺牲一部分容易混淆但在干净集上可被正确分类的模式，却提升局部稳定性。实验中 fgsm\_train 在 FGSM 指标上更强，说明内层扰动确实改变了模型对梯度方向的敏感性。

校准误差提供了另一类可靠性信息。一个准确率较高但过度自信的模型，在医疗、自动驾驶或安全审核场景中仍然危险，因为错误预测可能以高置信度输出。可靠性图把置信度分箱后比较经验准确率，能够直接观察过度自信或保守倾向。延迟指标则提醒我们，鲁棒方法不能只看指标提升，还要考虑部署成本。最终模型选择应根据应用风险权衡：低风险场景可偏向干净准确率，高风险场景应优先鲁棒性和校准。

## 额外实验的创新点

额外实验通过连接前面分类、对抗和可解释性内容形成统一评价：先在 CIFAR-10 上训练三组模型，再用相同扰动半径评估鲁棒性，最后用 ECE 和特征响应解释模型行为。这样可以回答三个问题：模型是否准确、模型在邻域扰动下是否稳定、模型的置信度是否可信。相比只报告攻击曲线，这一设计更接近实际模式识别系统的验收标准。

## 鲁棒优化的消融逻辑

额外实验采用 baseline、strong augmentation 和 FGSM training 三组对照，是为了把自然泛化和对抗鲁棒性分离。baseline 提供标准经验风险最小化基线：

$$\min_{\theta} \mathbb{E}_{(x,y)} \ell(f_{\theta}(x), y).$$

strong augmentation 把样本替换为  $t(x)$ ，其中  $t$  来自裁剪、翻转和颜色扰动等自然变换族：

$$\min_{\theta} \mathbb{E}_{(x,y), t \sim \mathcal{T}} \ell(f_{\theta}(t(x)), y).$$

FGSM training 则把变换族换成由模型梯度生成的最坏方向。三者的差异在于邻域定义不同：自然增强覆盖语义保持的图像变换，对抗训练覆盖损失上升最快的数值方向。本次结果中 strong\_aug 未显著优于 baseline，说明自然增强不能替代对抗邻域；若对抗训练提高 robust accuracy 同时降低 clean accuracy，则体现局部稳定性与干净样本拟合之间的取舍。

## 校准、置信阈值与部署解释

ECE 只度量分箱后的平均校准误差，还可以结合最大校准误差 MCE 和负对数似然 NLL 分析。若模型输出置信度  $q_i$ ，预测正确指示为  $r_i$ ，则可靠性图比较每个分箱内的  $\bar{q}_m$  与  $\bar{r}_m$ 。理想模型满足

$$\mathbb{P}(\hat{y} = y \mid \hat{p} = q) = q.$$

这比准确率更接近部署需求。例如一个 90% 准确率但总以 99% 置信度输出的模型，在人工复核系统中会错误压低复核优先级。若校准较好，可以设置置信阈值：低于阈值的样本进入人工复核，高于阈值的样本自动通过。鲁棒训练若同时改善 ECE，说明它不仅改善了局部稳定性，也可能降低过度自信。

推理延迟用于约束工程可行性。对抗训练通常不改变推理结构，因此单次推理延迟与 baseline 接近；若采用集成、随机平滑或多步攻击防御，延迟会显著增加。报告中把延迟与准确率、鲁棒性、ECE 放在同一表中，是为了避免只从单项指标选择模型。实际系统可以把目标写成加权效用：

$$U = \alpha A_{\text{clean}} + \beta A_{\text{robust}} - \gamma ECE - \lambda T,$$

其中  $T$  是延迟，权重由应用风险决定。高风险应用应提高  $\beta, \gamma$ ，低延迟边缘设备则需提高  $\lambda$ 。

## 可解释性与鲁棒性的关系

鲁棒模型常表现出更平滑的输入梯度。直观上，若分类结果依赖少数高频像素，沿这些像素的扰动即可快速改变 logit；若模型利用更大范围的主体结构，局部扰动必须同时改变多个稳定特征才会成功。可解释性图不能作为鲁棒性的充分证明，但能提供辅助证据：响应区域更集中于目标主体、背景响应降低，通常意味着模型减少了对非因果特征的依赖。实验五把响应图、鲁棒准确率和 ECE 放在同一叙述中，形成“行为指标—置信指标—解释证据”的三角验证。

需要强调的是，FGSM training 只能防御与训练半径相近的一步攻击。更强的 PGD 可写为

$$x^{t+1} = \Pi_{B_{\epsilon}(x)}(x^t + \alpha \text{sign}(\nabla_x \ell(f_{\theta}(x^t), y))),$$

它通过多步投影更充分地近似内层最大化。受时间和报告篇幅限制，额外实验采用 FGSM 作为可复现实验；若继续扩展，应加入 PGD、AutoAttack 和不同半径的鲁棒曲线，以检验防御是否只适配单一步长攻击。

## 误差条、半径曲线与未来扩展

鲁棒实验若进一步严格化，应报告不同随机种子下的均值与标准差。设第  $s$  个种子得到指标  $a_s$ ，则均值和标准误差为

$$\bar{a} = \frac{1}{S} \sum_s a_s, \quad SE = \sqrt{\frac{1}{S(S-1)} \sum_s (a_s - \bar{a})^2}.$$

正式结果采用 2026/2027/2028 三个随机种子，并以误差条展示结论稳定性。鲁棒准确率也不只在单个  $\epsilon$  上评价，而是给出  $A(\epsilon)$  曲线，其中  $\epsilon \in \{0, 1/255, 2/255, 4/255, 8/255\}$ 。曲线下面积越大，说明模型在更宽扰动范围内保持稳定。

额外实验还可以加入温度缩放改善校准。给定 logits  $z$ ，温度缩放使用  $p = \text{softmax}(z/T)$ ，在验证集上选择  $T$  最小化 NLL。 $T > 1$  会软化概率分布，通常降低过度自信。温度缩放不改变预测类别，因此 clean accuracy 不变，但 ECE 可能下降。若与对抗训练结合，可以区分“局部稳定性”和“概率校准”两类改进来源。该分析体现了额外实验的深度：鲁棒性、校准和解释性是可分解的评价维度，需要分别设计指标。

从类脑智能角度看，鲁棒识别还可联系到人类视觉的稳定性。人类对小幅噪声、局部遮挡和光照变化通常较稳定，是因为识别依赖多尺度结构和语义先验；标准 CNN 可能利用高频纹理捷径。对抗训练迫使模型在局部邻域内保持标签一致，相当于鼓励更平滑的判别函数。虽然这还远不能等同于生物视觉机制，但它为“准确率之外的智能可靠性”提供了可实验验证的切入点。

## 鲁棒实验的算法化描述

额外实验的核心步骤可概括为四步。第一，在相同数据划分上训练 baseline 与增强模型，保存干净测试准确率。第二，对每个测试 batch 打开输入梯度，按

$$\delta = \epsilon \text{sign}(\nabla_x \ell(f_{\theta}(x), y))$$

构造裁剪后的对抗样本。第三，在相同  $\epsilon$  下评估所有模型的 robust accuracy，保证比较公平。第四，收集预测置信度并按区间分箱，计算 ECE 和可靠性图。该流程把攻击、训练、评估和校准分开，使每个指标都有明确来源。

鲁棒训练的伪目标虽然写成 min-max，但实际 FGSM training 使用一次梯度近似内层最大化。若训练时直接在  $x$  上求梯度并更新模型，需要注意先清空参数梯度，再对输入开启 requires\_grad，攻击样本生成后应从计算图中分离，随后再进行模型参数更新。否则可能出现二阶图残



留或梯度累积错误。报告没有展示运行流水账，但这些实现细节决定了鲁棒结果是否可信。

表 16: 鲁棒实验指标的含义。

| 指标      | 解释                  |
|---------|---------------------|
| Clean   | 原始测试集准确率，衡量标准识别能力。  |
| FGSM    | 固定扰动半径下准确率，衡量局部稳定性。 |
| ECE     | 置信度与经验准确率差异，衡量校准。   |
| Latency | 单图推理时间，衡量部署代价。      |

从结果选择角度看，若某模型 clean 略低但 FGSM 与 ECE 明显更好，在安全敏感任务中仍可能更优。相反，若应用只处理低风险离线分类，clean accuracy 权重可以更高。额外实验的价值就在于把这种取舍显式化，让模型选择从“谁的准确率高”转向“谁在给定风险约束下更合适”。

实验过程与心得

鲁棒实验一开始我只在单个随机种子和单个扰动半径上比较，结论看起来过于绝对。修正时我增加 3 个 seed 和 5 个  $\epsilon$ ，并同时记录 ECE 与延迟。调参中发现 FGSM 训练能明显提高攻击下准确率，但校准未必更好；强增强在 ECE 和耗时上更稳。最终我把结论改为多目标取舍。这个实验让我认识到，可靠性评价必须避免用单一指标替代系统判断。

跨实验综合评价

五个实验按“数值计算基础、深度学习训练、图像分类、检测与多任务视觉、鲁棒识别”展开。各部分的任务重点和完成内容如下表所示。

表 17: 各实验任务、重点与完成情况。

| 实验 | 任务重点            | 完成内容                                 |
|----|-----------------|--------------------------------------|
| 一  | Python/NumPy 基础 | 语法组织、ndarray、广播、向量化、图像数组和距离矩阵。       |
| 二  | PyTorch 基础链路    | 张量、DataLoader、自动微分、优化器、混淆矩阵和模型保存。    |
| 三  | CIFAR-10 分类     | CNN、BN/Dropout、数据增强、迁移学习、逐类错误和响应图。   |
| 四  | 检测与综合视觉         | PyTorch 目标检测/实例分割、迁移学习、FGSM、DCGAN。   |
| 五  | 额外鲁棒实验          | clean/robust accuracy、ECE、延迟与鲁棒训练解释。 |

从数学结构看，实验反复出现三类对象。第一类是表示结构，包括 ndarray、距离矩阵、卷积特征、候选框、mask 和潜变量，它们决定了数据如何进入模型。第二类是优化结构，包括交叉熵、SGD/Adam、迁移学习线性头、GAN 极小极大和鲁棒 min-max，它们决定了参数更新方向。第三类是评价结构，包括混淆矩阵、IoU、AP、ECE、攻击曲线和延迟，它们决定了实验结果如何被解释。

从误差分析看，不同实验的错误来源具有层次性。NumPy 实验的错误常来自 dtype、shape、广播轴和视图/复制；PyTorch 实验的错误常来自学习率、batch size、训练/评估状态和数据归一化；CIFAR-10 分类的错误常来自类间相似、主体过小、背景偏置和训练预算；检测实验的错误可分解为候选框遗漏、NMS 阈值、框回归偏移和 mask 轮廓不准；鲁棒实验的错误则与局部梯度、过度自信和扰动半径有关。

课程知识可以概括为“表示—学习—决策”的闭环。表示层面，数组、卷积特征、候选框和潜变量把原始数据映射到可计算空间；学习层面，监督学习、迁移学习、对抗学习和生成对抗学习分别对应不同损失；决策层面，分类阈值、NMS、置信度、鲁棒半径和延迟共同影响模型使用。五个实验在同一闭环中逐步增加任务复杂度。

若把五个实验抽象为统一优化问题，可以写成

$$\min_{\theta} \mathbb{E}_{(x,y) \sim \mathcal{D}} [\ell(f_{\theta}(x), y) + \lambda R(\theta, x)],$$

其中  $R$  在不同实验中含义不同：在 NumPy 实验中对内存和计算代价，在 PyTorch/FashionMNIST 中对应正则

化，在 CIFAR-10 中对应增强或迁移先验，在检测任务中对应框回归和 mask 约束，在鲁棒实验中对邻域最坏损失。该统一视角把基础编程、深度学习、检测、生成和鲁棒性联系到同一个风险最小化框架中。

局限性与扩展路线

本次实验使用可复现子集和统一训练预算完成多任务链路，因此分类准确率、检测 IoU 和生成样本质量仍应理解为当前硬件和时间预算下的结果。若扩展实验一和实验二，可进一步加入更大规模矩阵乘、分块近邻检索、混合精度训练和学习率调度，分析计算效率与收敛稳定性的关系。

图像分类实验可继续扩展到余弦退火、Mixup/CutMix、标签平滑和更强骨干网络，并用 Grad-CAM 或特征嵌入检查模型是否关注主体区域。检测实验可进一步补充 COCO 风格  $AP_{75}$ 、mask AP、多尺度增强和 NMS 阈值消融。对抗实验可加入 PGD、AutoAttack 和随机平滑，检验防御是否只适配一步攻击。生成实验可补充 FID、潜变量插值和模式覆盖分析，避免只依据少量生成样本做结论。

类脑智能相关的进一步分析可围绕稳健表示和不确定性展开。人类视觉通常综合形状、上下文和经验先验，对局部噪声较稳定；深度模型容易学习纹理捷径和背景相关性。后续可比较标准训练、强增强、对抗训练和校准方法在可解释图上的差异，分析模型是否从局部高频响应转向更稳定的主体结构响应。

综合结论

本报告完成了从 Python/NumPy 基础、PyTorch 自动微分、CIFAR-10 图像分类，到迁移学习、检测、对抗攻击、生成建模和鲁棒识别的完整实验链路。实验一说明矩阵化表达和内存布局决定数值程序效率；实验二给出深度学习训练闭环和梯度推导；实验三展示卷积归纳偏置、增强和迁移学习对视觉分类的影响；实验四把识别扩展到结构化输出、安全可靠性和分布生成；实验五进一步从鲁棒性和校准角度审查模型可靠性。由此可见，模式识别系统的质量不能只由单一准确率衡量，还必须同时报告数据流、优化过程、误差结构、鲁棒性、可解释证据和复现条件。

符号与指标速览

报告中反复出现的符号可归纳为四组： $x, y$  表示样本与标签， $\theta$  表示模型参数， $\ell$  表示损失函数， $\delta$  表示输入扰动。分类实验关注  $p(y|x)$  与交叉熵，检测实验关注目标框  $B$  与置信排序，生成实验关注潜变量  $z$  到图像  $G(z)$  的映射，鲁棒实验关注  $B_{\epsilon}(x)$  邻域内的最坏损失。统一符号有助于把多个实验的公式放在同一框架下理解。

表 18: 报告中关键指标的解释。

| 指标       | 含义                    |
|----------|-----------------------|
| Acc.     | 分类正确比例，适合总体性能概览。      |
| IoU/AP50 | 定位重合度与检测排序质量，衡量结构化输出。 |
| ECE      | 置信度与经验准确率差异，衡量概率校准。   |
| Latency  | 单样本推理时间，衡量部署代价。       |

这些指标互相补充。准确率高但 ECE 大，说明模型可能过度自信；IoU 高但 recall 低，说明命中目标定位较好但漏检仍多；鲁棒准确率高但延迟大，说明部署时需要权衡资源。最终模型评价应从任务风险出发，把性能、可靠性和代价共同纳入判断。

参考文献

[1] Python Software Foundation. Python 3 Documentation.  
[2] C. R. Harris et al., "Array programming with NumPy," Nature, 2020.



- [3] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," NeurIPS, 2019.
- [4] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST," arXiv:1708.07747, 2017.
- [5] A. Krizhevsky, "Learning Multiple Layers of Features from Tiny Images," 2009.
- [6] K. He et al., "Deep Residual Learning for Image Recognition," CVPR, 2016.
- [7] K. He et al., "Mask R-CNN," ICCV, 2017.
- [8] S. Ren et al., "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," NeurIPS, 2015.
- [9] I. Goodfellow et al., "Explaining and Harnessing Adversarial Examples," ICLR, 2015.
- [10] A. Madry et al., "Towards Deep Learning Models Resistant to Adversarial Attacks," ICLR, 2018.
- [11] A. Radford et al., "Unsupervised Representation Learning with Deep Convolutional GANs," ICLR, 2016.
- [12] R. R. Selvaraju et al., "Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization," ICCV, 2017.
- [13] C. Guo et al., "On Calibration of Modern Neural Networks," ICML, 2017.
- [14] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," ICLR, 2015.
- [15] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," ICML, 2015.
- [16] N. Srivastava et al., "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," JMLR, 2014.

## 核心代码贴片与复现

### 实验一：NumPy 向量化与分块。

```
def pairwise_distance(X, Z, block=512):
    out = np.empty((len(X), len(Z)), dtype=np.float32)
    z2 = (Z * Z).sum(axis=1)[None, :]
    for s in range(0, len(X), block):
        xb = X[s:s + block]
        x2 = (xb * xb).sum(axis=1)[:, None]
        out[s:s + block] = x2 + z2 - 2 * xb @ Z.T
    return np.maximum(out, 0)
```

### 实验二/三：PyTorch 训练与模型对照。

```
def train_epoch(model, loader, optimizer, criterion, device):
    model.train()
    for x, y in loader:
        x, y = x.to(device), y.to(device)
        optimizer.zero_grad(set_to_none=True)
        loss = criterion(model(x), y)
        loss.backward()
        optimizer.step()

configs = [
    ("cnn", SmallCNN()),
    ("cnn_bn_aug", SmallCNN(batch_norm=True, dropout=0.25)),
    ("resnet18_head", frozen_resnet18(num_classes=10)),
]
for name, model in configs:
    history[name] = fit_and_eval(model, train_loader, test_loader)
```

### 实验四：Faster R-CNN 与 AP50。

```
def pennfudan_target(mask):
    ids = np.unique(mask)[1:]
    masks = mask[None] == ids[:, None, None]
    boxes = masks_to_boxes(torch.as_tensor(masks, dtype=torch.uint8))
    return {"boxes": boxes, "labels": torch.ones(len(ids), dtype=torch.int64)}

model.roi_heads.nms_thresh = 0.5
for image, target in test_set:
    pred = model([image.to(device)])[0]
    pred = filter_by_score(pred, threshold=0.5)
    for box, score in sorted(zip(pred["boxes"], pred["scores"]), key=lambda z: -z[1]):
        match = best_unmatched_iou(box.cpu(), target["boxes"])
        if match.iou >= 0.5:
            tp += 1; mark_used(match.gt_id)
        else:
            fp += 1
fn = len(all_gt) - tp
precision, recall = tp / (tp + fp), tp / (tp + fn)
ap50 = integrate_pr_curve(all_detections, all_gt, iou_thr=0.5)
```

### 实验五：FGSM、ECE 与多目标取舍。

```
def fgsm_batch(model, x, y, eps):
    x = x.detach().clone().requires_grad_(True)
    loss = F.cross_entropy(model(x), y)
    loss.backward()
    return torch.clamp(x + eps * x.grad.sign(), 0, 1)

def ece(conf, ok, bins=10):
    err = 0.0
    for lo, hi in zip(np.linspace(0, 1, bins, endpoint=False), np.linspace(0.1, 1, bins)):
        m = (conf > lo) & (conf <= hi)
        if m.any():
            err += m.mean() * abs(ok[m].mean() - conf[m].mean())
    return err

for seed in [2026, 2027, 2028]:
    for eps in [0, 1/255, 2/255, 4/255, 8/255]:
        robust_curve[seed, eps] = eval_fgsm(model, test_loader, eps)
```

复现入口为 `.\run_all.ps1 -Stage all -Profile full -Device auto -Seed 2026`。核心代码位于 `submission/src/experiments.py`、`figures.py` 与 `build_report.py`；数据下载、训练、绘图、报告编译和检查均由统一脚本串联。公开仓库：[github.com/JustinZHAO-05/pattern-recognition-brain-like-intelligence-experiments](https://github.com/JustinZHAO-05/pattern-recognition-brain-like-intelligence-experiments)。